SQL Indexes

Chapter 05 - Sub-Chapter 02

SQL Databases Series

Why Indexes?	B-Tree internals, seq scan vs index scan
Index Types	B-Tree, Hash, GIN, GiST, BRIN, Partial
Compound & Covering	Column order, INCLUDE, index-only scans
Partial & Expression	Index subsets and function results
Maintenance & Monitoring	Bloat, unused indexes, REINDEX, JPA

1 Why Indexes?

The difference between milliseconds and minutes

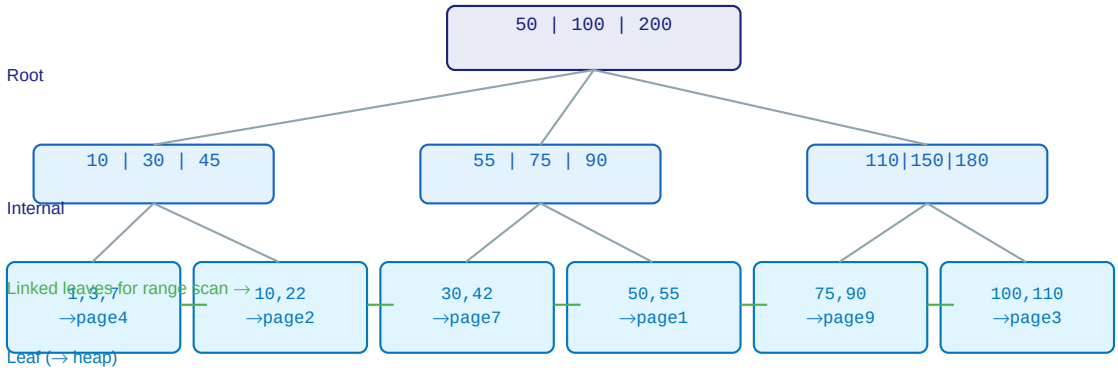
Imagine a library with a million books — no catalog, no shelf numbers. Finding one book means checking every single book. That is what a database does without an index: a **sequential scan** — reading every row, one by one.

An index is a separate data structure that maps column values to row locations. Like a library catalog: look up 'Tolkien' and get 'Shelf 7, Row 3' instantly. With an index, finding a row among 10 million takes 3-4 disk reads instead of 10 million.

Real performance difference (PostgreSQL, 10M row orders table):

```
-- Without index on user_id:
EXPLAIN ANALYZE SELECT * FROM orders WHERE user_id = 42;
-- Seq Scan on orders  (cost=0.00..312485.00 rows=127)
-- Execution Time: 2847.231 ms   <-- nearly 3 seconds!

-- After: CREATE INDEX idx_orders_user_id ON orders(user_id);
EXPLAIN ANALYZE SELECT * FROM orders WHERE user_id = 42;
-- Index Scan using idx_orders_user_id  (cost=0.56..528.43 rows=127)
-- Execution Time: 1.834 ms      <-- 1500x faster!
```

B-Tree Index — How PostgreSQL Finds Rows Without Scanning All Data

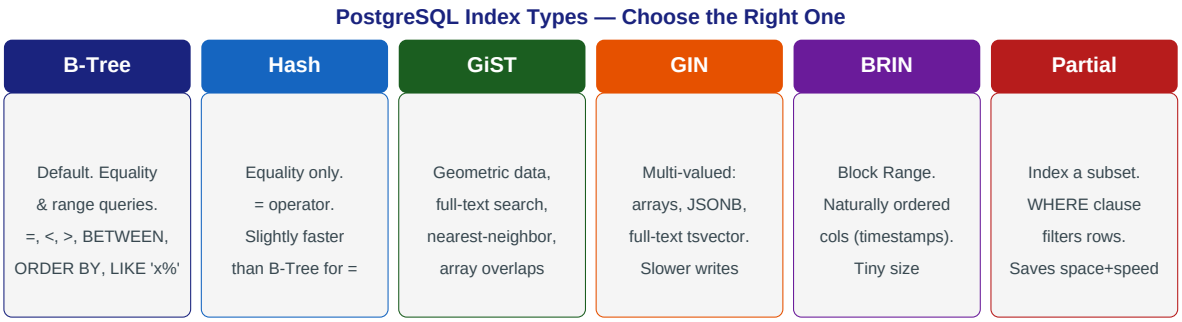


B-Tree height ~3-4 levels for millions of rows — O(log n) lookup vs O(n) full scan

The trade-off: Indexes speed up reads but slow down writes. Every INSERT, UPDATE, or DELETE must also update every index on that table. A table with 10 indexes pays 10x the write overhead. Index what you query, not everything.

2 Index Types

B-Tree, Hash, GIN, GiST, BRIN, Partial

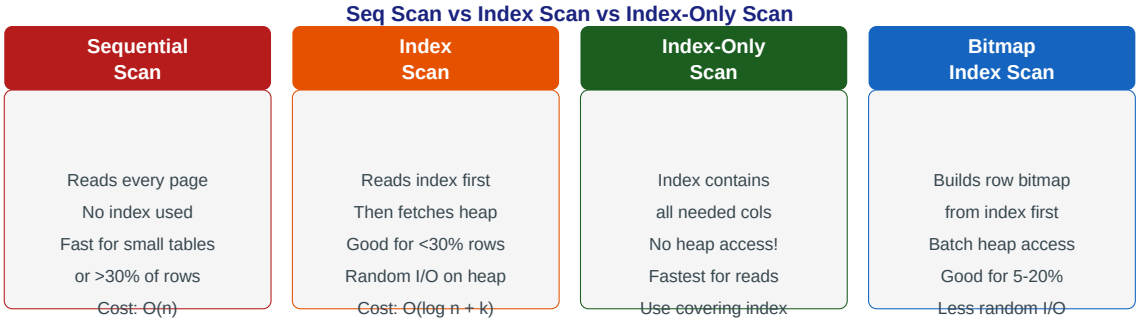


Creating the right index for the job:

```
-- B-Tree (default) -- equality, range, sorting
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_created ON orders(created_at DESC);  -- ordered
CREATE INDEX idx_users_email    ON users(email);

-- Hash -- equality only, slightly faster than B-Tree for pure =
CREATE INDEX idx_sessions_token ON sessions USING HASH (session_token);

-- GIN -- for JSONB and array queries (containment, existence)
CREATE INDEX idx_products_tags  ON products USING GIN (tags);     -- tags is array
CREATE INDEX idx_orders_meta    ON orders   USING GIN (metadata); -- metadata is JSONB

-- GIN for full-text search
ALTER TABLE articles ADD COLUMN search_vec TSVECTOR
    GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || body)) STORED;
CREATE INDEX idx_articles_fts ON articles USING GIN (search_vec);

-- Query: full-text search
SELECT title FROM articles
WHERE search_vec @@ to_tsquery('english', 'postgresql & index');

-- BRIN (tiny index for naturally ordered large tables)
-- Orders table: order_id is sequential, timestamps are naturally ordered
CREATE INDEX idx_orders_created_brin ON orders USING BRIN (created_at);
-- BRIN index on 100M rows = ~256KB vs B-Tree's ~3GB!
```

3 Compound and Covering Indexes

Multi-column indexes and index-only scans



Compound (multi-column) indexes — column order matters critically:

A compound index on (a, b, c) can serve queries filtering on a, (a, b), or (a, b, c) — but NOT queries filtering only on b or c. Think of it like a phone book: sorted by last name then first name. You can look up by last name alone, but not first name alone.

```
-- Query: find confirmed orders for a user, sorted by date
SELECT * FROM orders
WHERE user_id = 42 AND status = 'CONFIRMED'
ORDER BY created_at DESC;

-- Wrong index order (can't use both filter efficiently):
CREATE INDEX idx_wrong ON orders(status, user_id, created_at);
-- Optimizer uses this for status filter but not user_id efficiently

-- Right index order: most selective / most used first
CREATE INDEX idx_orders_user_status_date
    ON orders(user_id, status, created_at DESC);
-- This serves: WHERE user_id=?
--              WHERE user_id=? AND status=?
--              WHERE user_id=? AND status=? ORDER BY created_at DESC
-- All three queries use this single index!
```

Covering index — eliminate heap access entirely:

An **index-only scan** occurs when all columns needed by a query are stored in the index itself. No heap (table) access required — just the index. This is often 2-5x faster than a regular index scan.

```
-- Query: dashboard shows order_id, status, total for user 42
SELECT order_id, status, total
FROM orders
WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 20;

-- Regular index: finds rows by user_id, then fetches heap for status/total
CREATE INDEX idx_orders_user ON orders(user_id);
-- Plan: Index Scan → fetch each row from heap (random I/O)

-- Covering index: INCLUDE the extra columns in the index
CREATE INDEX idx_orders_user_covering
    ON orders(user_id, created_at DESC)
    INCLUDE (order_id, status, total);  -- stored in leaf nodes of index
-- Plan: Index Only Scan -- no heap access at all!
```

```
-- Verify with EXPLAIN:
EXPLAIN SELECT order_id, status, total FROM orders
WHERE user_id=42 ORDER BY created_at DESC LIMIT 20;
-- "Index Only Scan using idx_orders_user_covering"  <--
```

4 Partial and Expression Indexes

Index only what you query

Partial index — index a subset of rows:

If you almost always query for a specific subset of rows (e.g., only 'PENDING' orders), a partial index covers only those rows. The index is smaller, fits in memory better, and queries are faster.

```
-- Only 2% of orders are PENDING at any time, but they're queried constantly
-- (dashboard, background jobs, notifications)

-- Full index: 10M orders × 50 bytes = 500MB
CREATE INDEX idx_orders_status ON orders(status);

-- Partial index: only PENDING orders — maybe 200K rows = 10MB!
CREATE INDEX idx_orders_pending ON orders(created_at)
WHERE status = 'PENDING';

-- This query uses the partial index (fast, tiny memory footprint):
SELECT order_id, user_id, created_at FROM orders
WHERE status = 'PENDING'
ORDER BY created_at ASC;

-- Another practical example: index only active users
CREATE INDEX idx_users_active_email ON users(email)
WHERE status = 'active';
-- Deleted/suspended users (maybe 30%) are excluded from the index
```

Expression index — index the result of a function:

```
-- Problem: case-insensitive email lookup is slow
SELECT * FROM users WHERE LOWER(email) = LOWER('Alice@Example.COM');
-- Without index: Seq Scan (can't use regular index on email)

-- Solution: expression index on LOWER(email)
CREATE INDEX idx_users_email_lower ON users(LOWER(email));

-- Now the query is fast:
SELECT * FROM users WHERE LOWER(email) = 'alice@example.com';
-- Uses index: idx_users_email_lower

-- Expression index on JSONB field
CREATE INDEX idx_orders_city ON orders((metadata->>'shipping_city'));
-- Supports: WHERE metadata->>'shipping_city' = 'New York'

-- Expression index for date part
CREATE INDEX idx_orders_year_month ON orders(DATE_TRUNC('month', created_at));
-- Supports: WHERE DATE_TRUNC('month', created_at) = '2024-01-01'
```

5 Index Maintenance and Monitoring

Bloat, unused indexes, VACUUM

Indexes do not maintain themselves. Over time they grow bloated from UPDATEs and DELETEs (dead tuples accumulate). PostgreSQL's autovacuum reclaims this space, but high-write tables may need manual attention. More critically: always monitor whether indexes are actually being used.

Find unused indexes (candidates for removal):

```
-- Indexes that have never been used since last stats reset
SELECT schemaname, tablename, indexname,
       idx_scan AS times_used,
       pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
JOIN pg_index USING (indexrelid)
WHERE idx_scan = 0           -- never used!
  AND NOT indisprimary       -- keep PKs
  AND NOT indisunique        -- keep UNIQUE indexes
ORDER BY pg_relation_size(indexrelid) DESC;
-- Drop unused indexes -- they slow down writes for no benefit

-- Find duplicate/redundant indexes
SELECT a.indexname AS idx1, b.indexname AS idx2, a.tablename
FROM pg_indexes a JOIN pg_indexes b
    ON a.tablename = b.tablename AND a.indexname < b.indexname
WHERE a.indexdef LIKE b.indexdef || '%';   -- b is a prefix of a
```

Rebuild a bloated index without locking the table:

```
-- Check index bloat
SELECT indexname,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size,
       idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
WHERE tablename = 'orders'
ORDER BY pg_relation_size(indexrelid) DESC;

-- Rebuild index CONCURRENTLY (no table lock, safe in production)
REINDEX INDEX CONCURRENTLY idx_orders_user_id;

-- Or rebuild all indexes on a table concurrently
REINDEX TABLE CONCURRENTLY orders;

-- Ensure autovacuum keeps up (tune for high-write tables)
ALTER TABLE orders SET (
    autovacuum_vacuum_scale_factor = 0.01,   -- vacuum after 1% of rows change
    autovacuum_analyze_scale_factor = 0.005  -- analyze after 0.5% change
);
```

Spring JPA — mapping indexes via annotations:

```
import jakarta.persistence.*;
import org.hibernate.annotations.Index;

@Entity
@Table(name = "orders",
    indexes = {
        @Index(name = "idx_orders_user_id",    columnList = "user_id"),
        @Index(name = "idx_orders_user_status", columnList = "user_id, status"),
```

```
        @Index(name = "idx_orders_created",    columnList = "created_at DESC")
    })
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long orderId;

    @Column(nullable = false)
    private Long userId;

    @Column(nullable = false, length = 20)
    private String status;

    @Column(nullable = false)
    private java.time.Instant createdAt;
    // ...
}
// Hibernate generates CREATE INDEX statements on schema creation
```

Key Rules

- Run EXPLAIN ANALYZE on every slow query -- look for Seq Scan on large tables.
- Index foreign key columns (user_id, order_id in child tables) -- crucial for JOIN performance.
- Use INCLUDE columns for index-only scans when a query always selects a fixed set of columns.
- Partial indexes: if a query always filters by a fixed value (status='PENDING'), index only those rows.
- Never blindly add indexes -- measure before and after, and drop unused ones regularly.
- REINDEX CONCURRENTLY to rebuild bloated indexes -- never block production tables.